

# UNIT-9

## Remote user authentication



**SUBSCRIBE**

# Outline

---

- Remote user authentication with symmetric encryption
- Remote user authentication with asymmetric encryption
- Kerberos

# Remote user authentication with symmetric encryption

---

## Mutual Authentication

1. A ---> KDC

$ID_a || ID_b || N1$

2. KDC ---> A

$E(K_a, [K_s || ID_b || N1 || E(K_b, [K_s || ID_a])])$

3. A ---> B

$E(K_b, [K_s || ID_a])$

4. B ---> A

$E(K_s, N2)$

5. A ---> B

$E(K_s, f(N2))$

# Remote user authentication with symmetric encryption

---

## One-way Authentication

1. A ---> KDC

$ID_a || ID_b || N_1$

2. KDC ---> A

$E(K_a, [K_s || ID_b || N_1 || E(K_b, [K_s || ID_a])])$

3. A ---> B

$E(K_b, [K_s || ID_a]) || E(K_s, M)$

# Kerberos

---

- **Kerberos** provides a trusted third-party authentication service that enables clients and servers to establish **authenticated communication**.
- **Kerberos** is an authentication service designed for use in a distributed environment.
- Version 4 of Kerberos makes use of **DES**, to provide the authentication service.

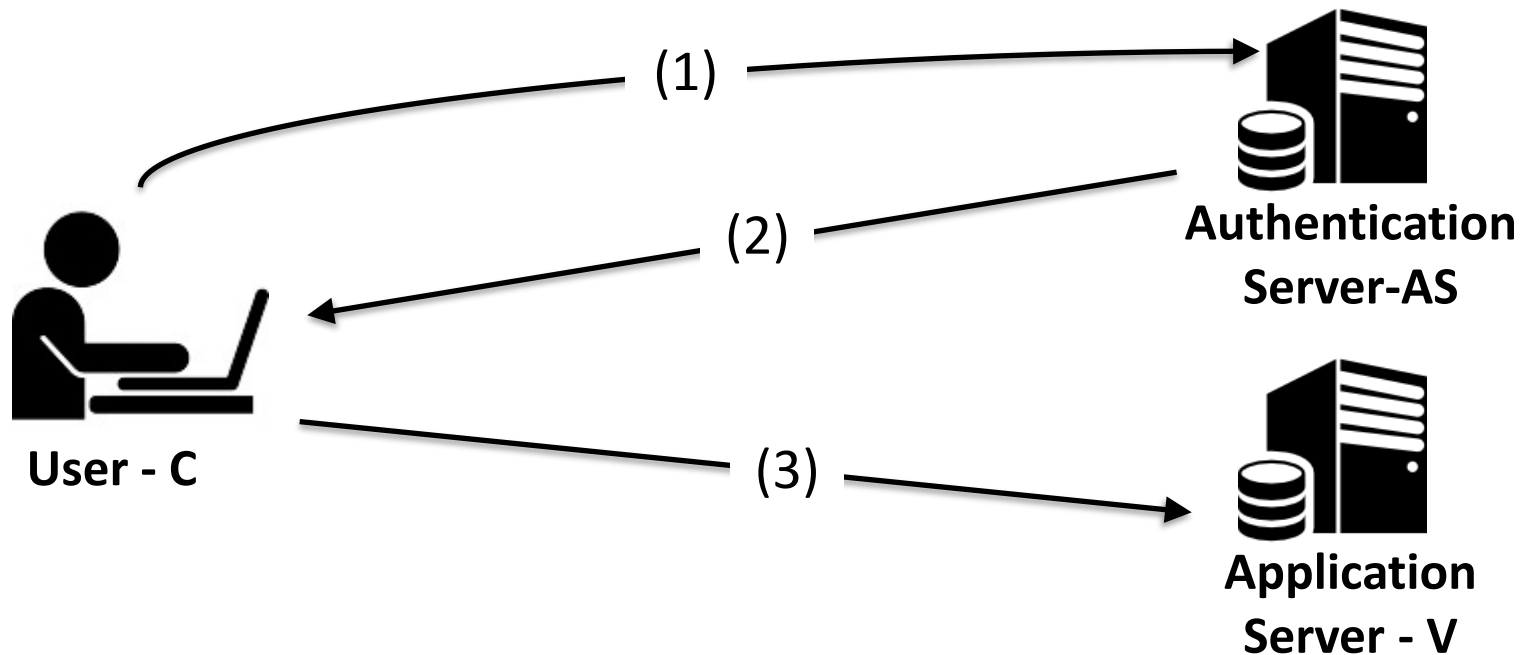
# Kerberos – Simple Dialogue

1. C ---> AS: **ID<sub>c</sub> || P<sub>c</sub> || ID<sub>v</sub>**

2. AS ---> C: **Ticket**

3. C ---> V: **ID<sub>c</sub> || Ticket**

**Ticket = E(K<sub>v</sub>, [ID<sub>c</sub>, A<sub>Dc</sub>, ID<sub>v</sub>])**



# Kerberos – Simple Dialogue

---

1. C ---> AS: **ID<sub>c</sub> || P<sub>c</sub> || ID<sub>v</sub>**

2. AS ---> C: **Ticket**

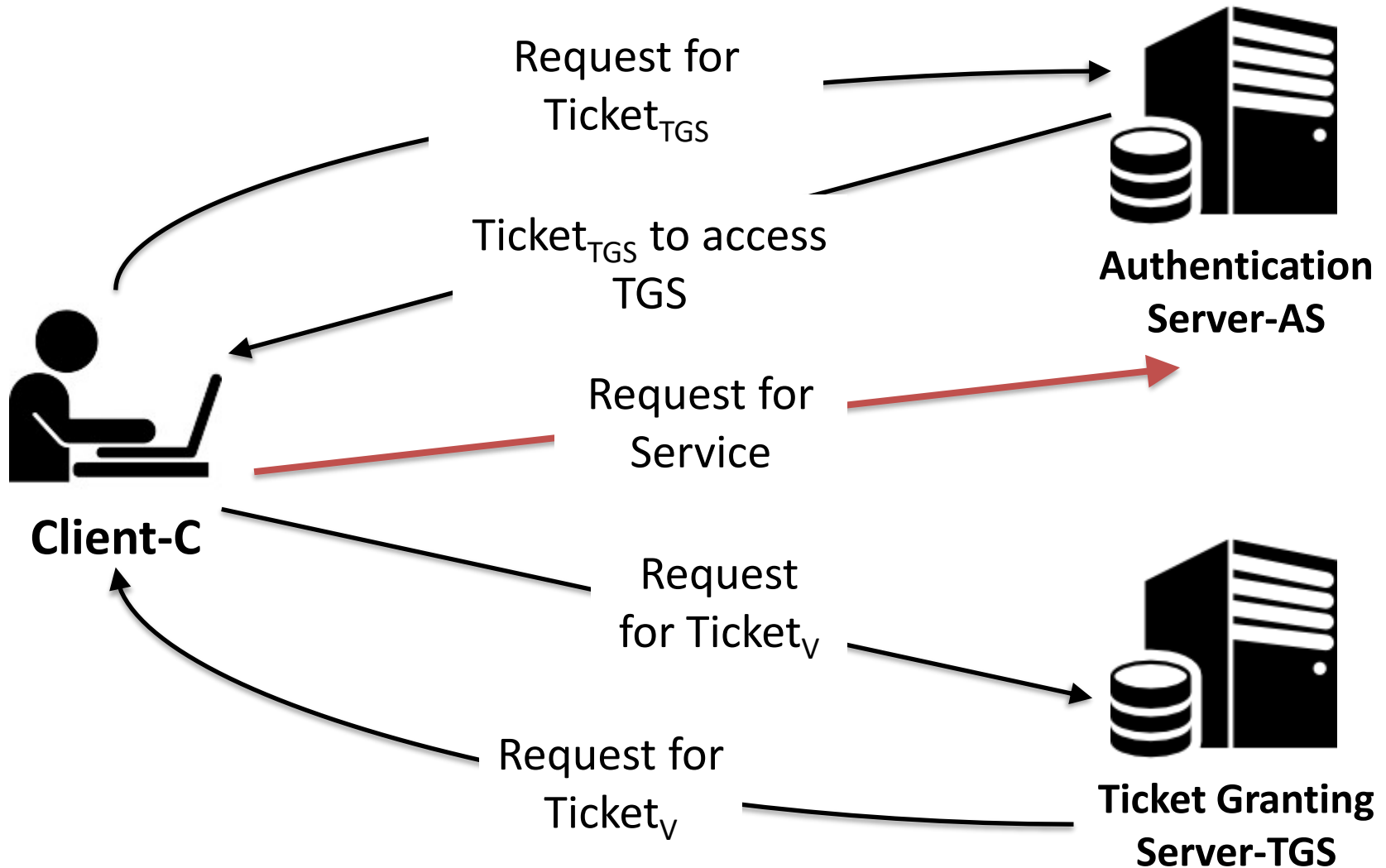
3. C ---> V: **ID<sub>c</sub> || Ticket**

$$\text{Ticket} = E(K_v, [ID_c, A_{Dc}, ID_v])$$

■ Where,

- C = Client
- AS = Authentication Server
- V = Server
- ID<sub>c</sub> = Identification of user C
- ID<sub>v</sub> = Identification of V
- P<sub>c</sub> = Password on User C
- A<sub>dc</sub> = Network Address of C
- K<sub>v</sub> = Secret key shared by AS and V

# Kerberos – More Secure Dialogue





# Kerberos – More Secure Dialogue

---

- Once per user logon session

1. C ---> AS:  $ID_c || ID_{TGS}$
2. AS ---> C:  $E(K_c, Ticket_{TGS})$

- Once per type of service

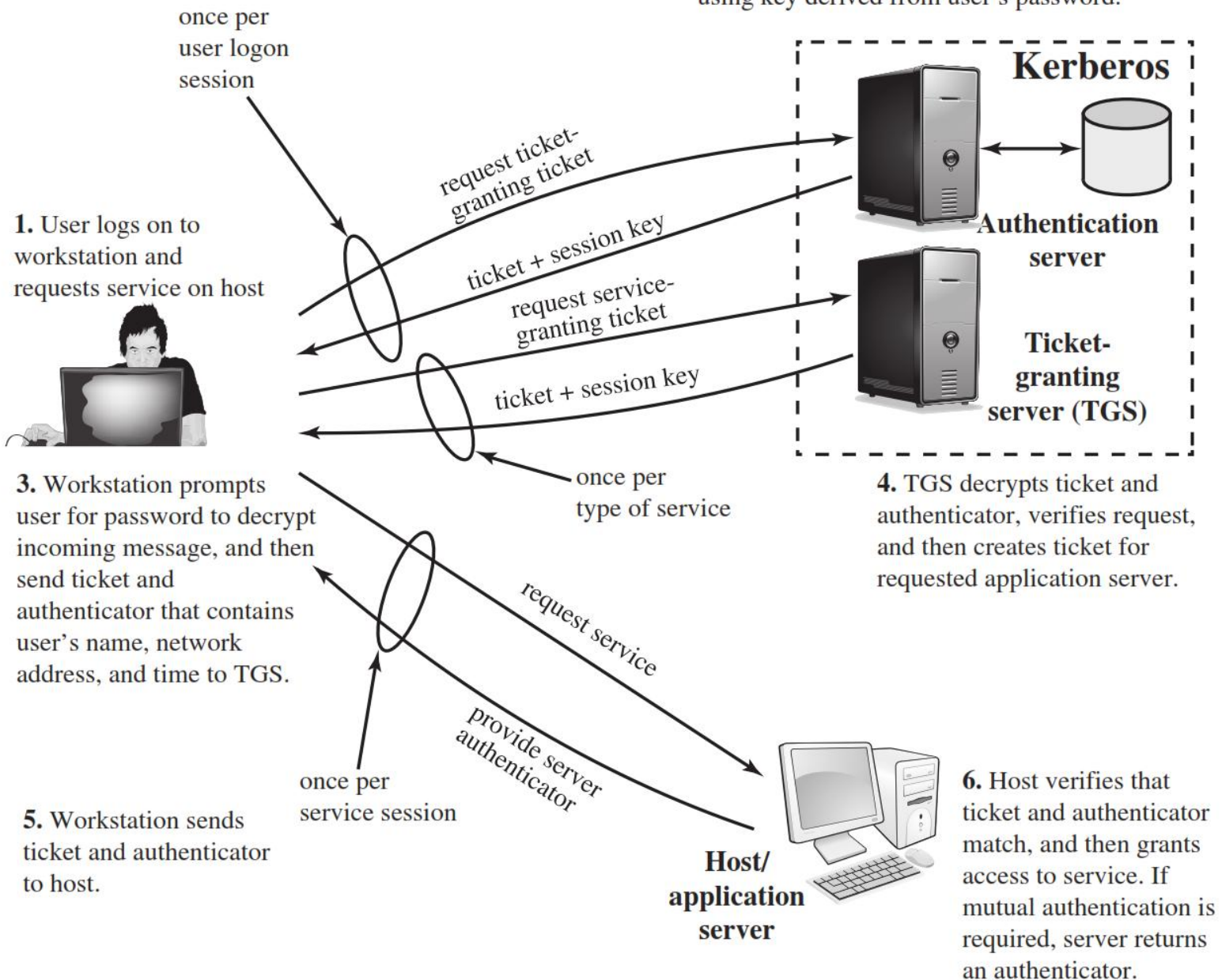
1. C ---> TGS:  $ID_c || ID_v || Ticket_{TGS}$
2. TGS ---> C:  $Ticket_v$

- Once per service session

1. C ---> V:  $ID_c || Ticket_v$

$Ticket_{TGS} = E(K_{TGS}, [ID_c, AD_c, Id_{TGS}, TS1, Lifetime1])$

$Ticket_v = E(K_v, [ID_c, AD_c, Id_v, TS2, Lifetime2])$

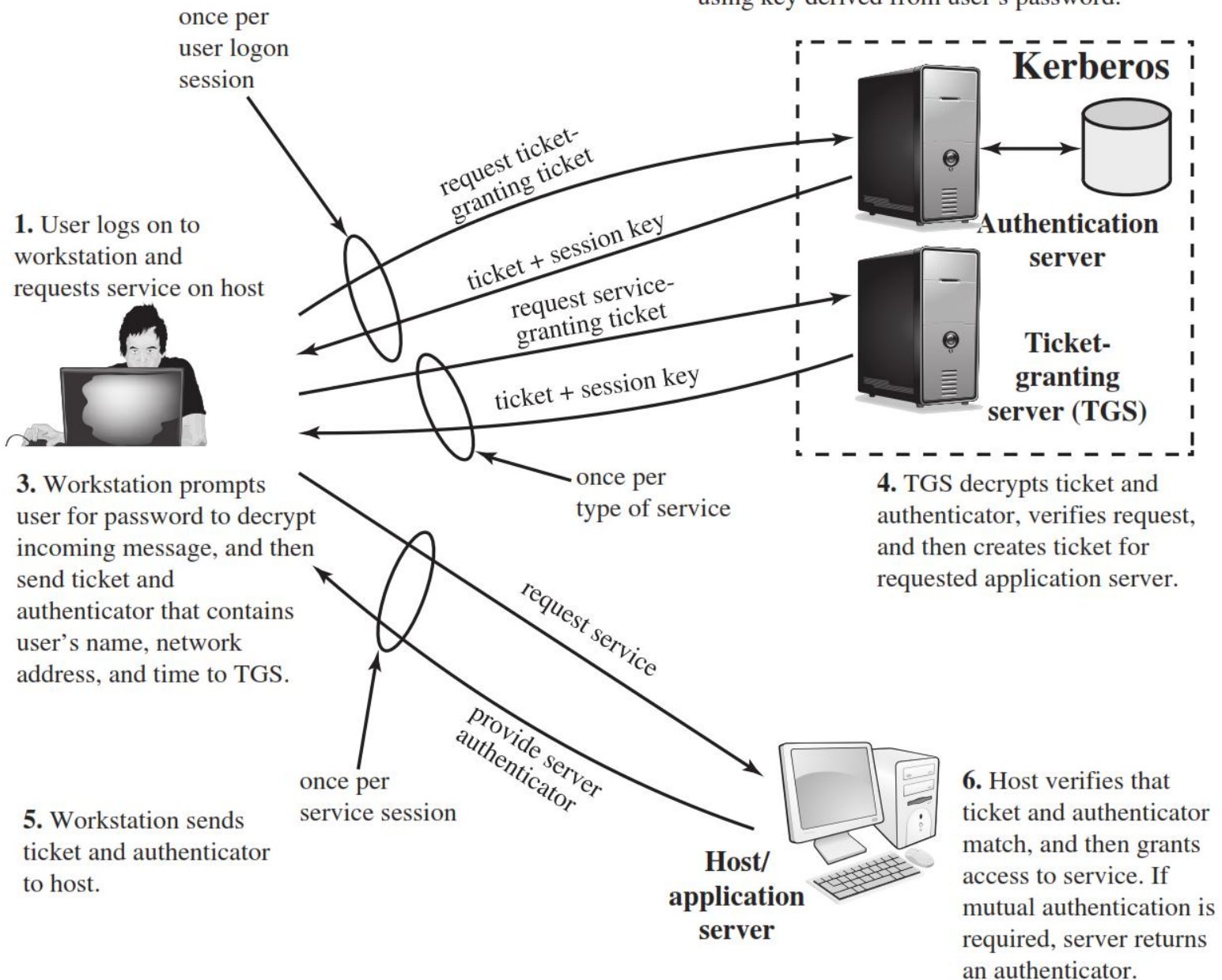


# Kerberos Version 4 – Step-1

---

- The client sends a *plaintext* request to the **AS** asking for a ticket it can use to talk to the **TGS**.
- Request:
  - Login name
  - **TGS** name
- Since this request contains only well-known names, it does not need to be sealed.

C ---> AS : IDc || ID<sub>tgs</sub> || TS<sub>1</sub>



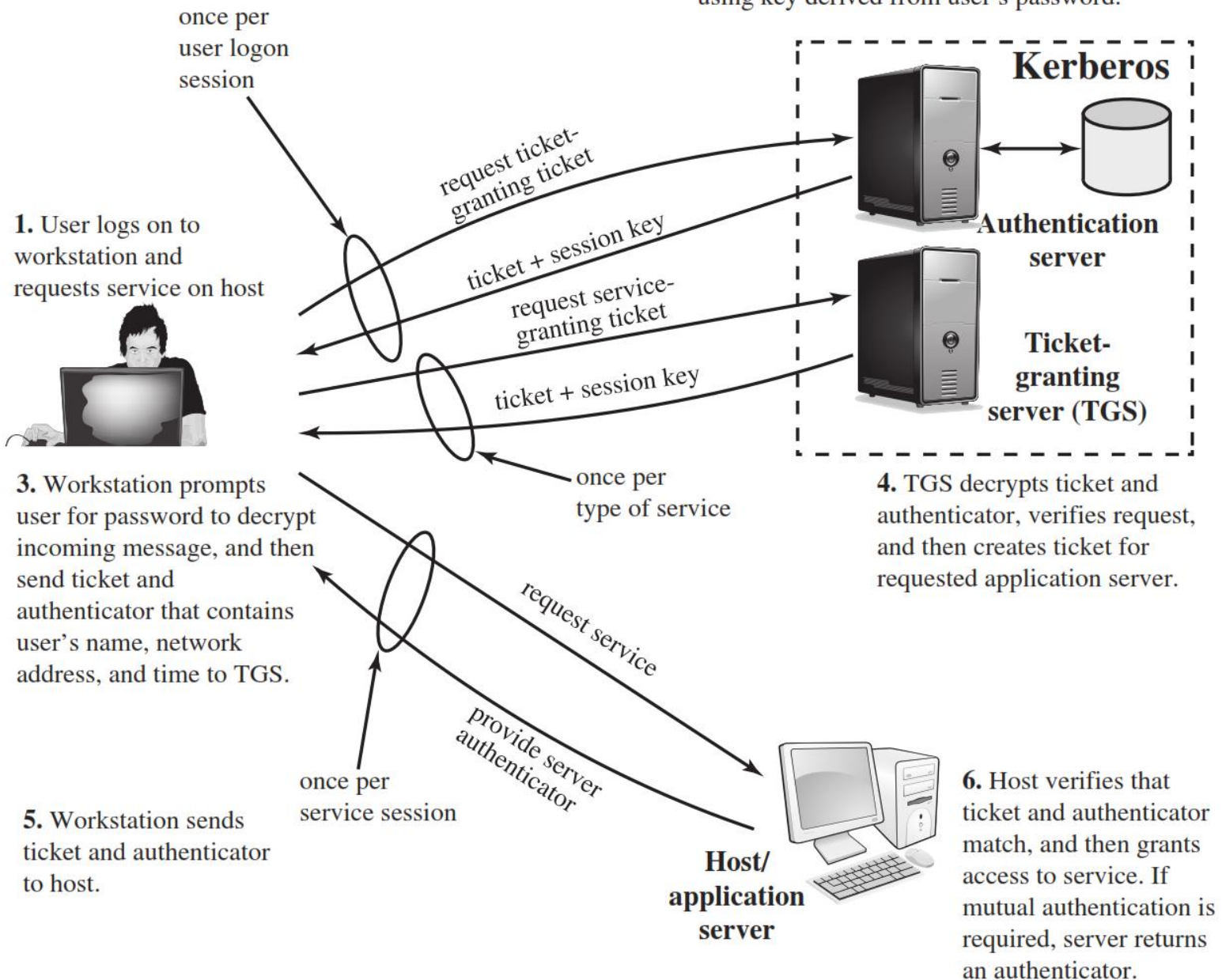
# Kerberos Version 4 – Step-2

- The **AS** finds the keys corresponding to the login name and the **TGS** name.
- The **AS** creates a ticket:
  - Login name
  - **TGS** name
  - Client network address
  - **TGS** session key
- The **AS** seals the ticket with the **TGS** secret key.

**AS--->C:  $E(K_c, [K_{c,tgs} || ID_{tgs} || TS_2 || Lifetime_2 || Ticket_{tgs}])$**

**$Ticket_{tgs} = E(K_{tgs}, [K_{c,tgs} || ID_c || AD_c || ID_{tgs} || TS_2 || Lifetime_2])$**



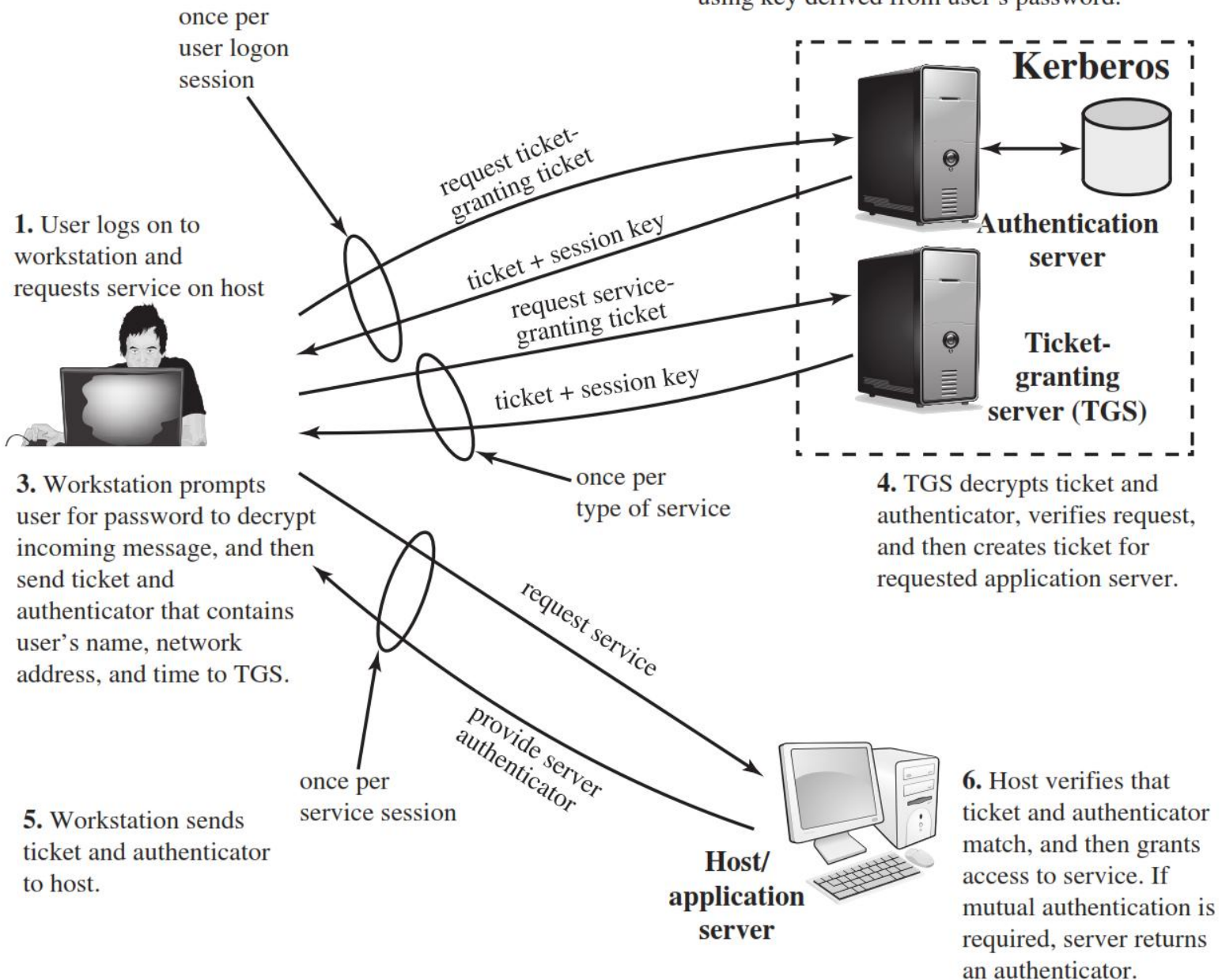


# Kerberos Version 4 – Step-3

---

- The client decrypts the message using the user's password as the secret key.
- The client now has a session key and ticket that can be used to contact the **TGS**.
- The client cannot see inside the ticket, since the client does not know the **TGS** secret key.
- When a client wants to start using a server (service), the client must first obtain a ticket.
- The client composes a request to send to the **TGS**

**C ---> TGS:  $Id_v || Ticket_{tgs} || Authenticator_c$**

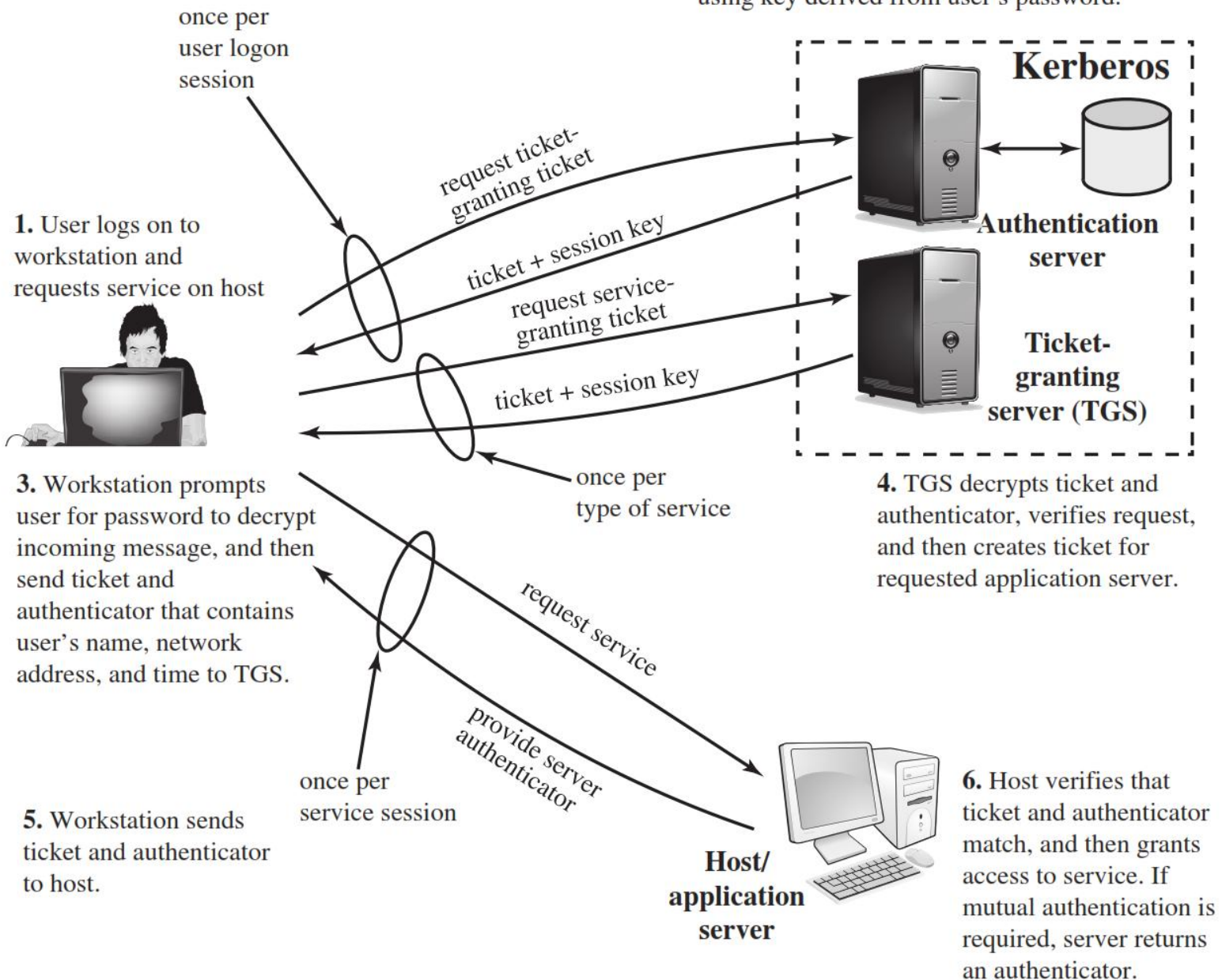




# Kerberos Version 4 – Step-4

- The **TGS** decrypts the ticket using its secret key. Inside is the TGS session key.
- The **TGS** decrypts the authenticator using the session key.
- The **TGS** checks to make sure login names, client addresses and **TGS** server name are all ok.
- **TGS** makes sure the authenticator is recent.
- Builds a ticket for the client and requested server. The ticket is sealed with the server key.
- Creates a session key
- Seals the entire message with the tgs session key and sends it to the client.

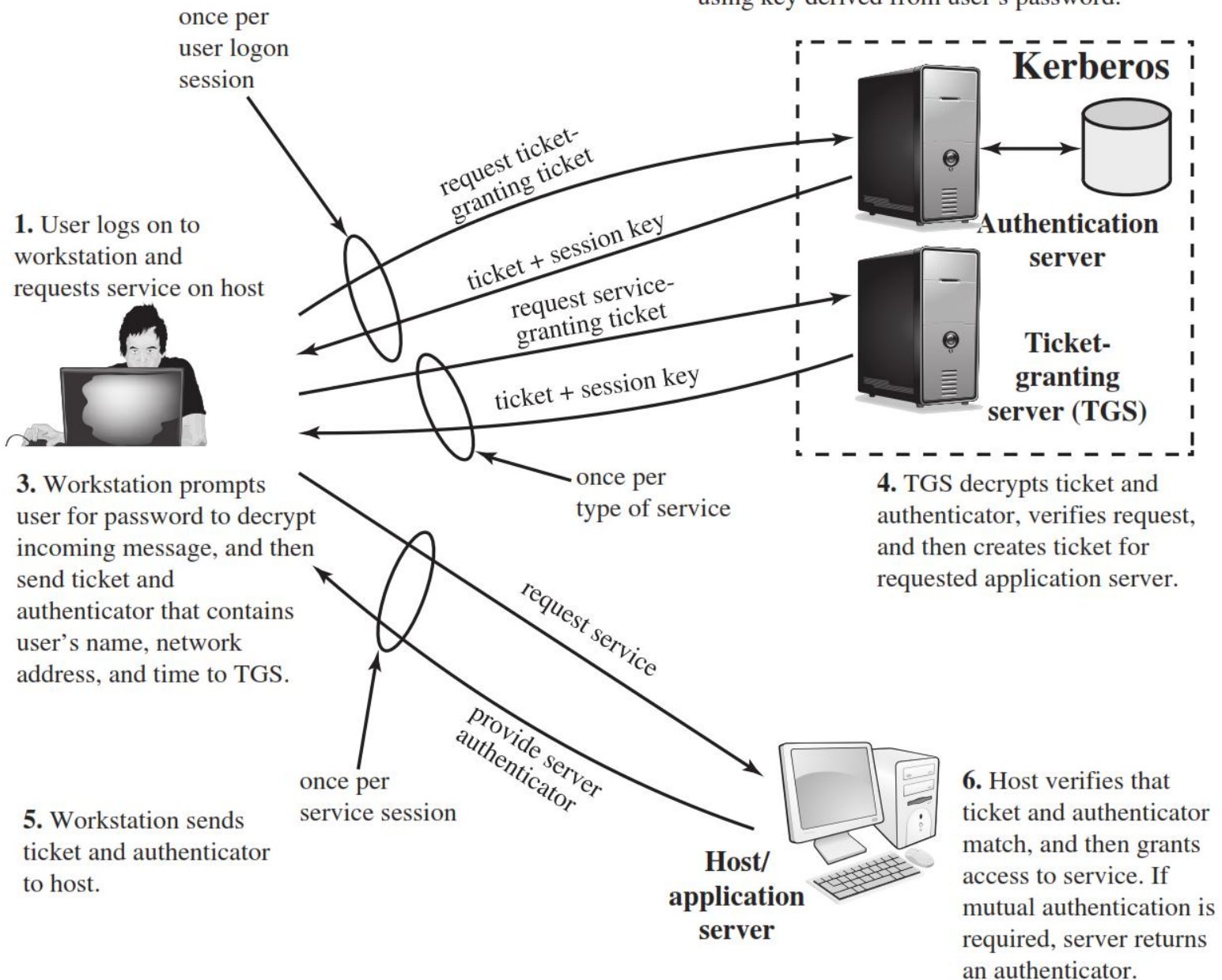
**TGS --->C:  $E(K_{c,tgs}, [K_{c,v} || ID_v || TS_4 || Ticket_v])$**



# Kerberos Version 4 – Step-4

---

$$\text{Ticket}_{\text{tgs}} = E(K_{\text{tgs}}, [K_{\text{c,tgs}} || \text{IDc} || \text{ADc} || \text{ID}_{\text{tgs}} || \text{TS}_2 || \text{Lifetime}_2])$$
$$\text{Ticket}_v = E(K_v, [K_{\text{c,v}} || \text{IDc} || \text{ADc} || \text{ID}_v || \text{TS}_4 || \text{Lifetime}_4])$$
$$\text{Authenticator}_c = E(K_{\text{c,tgs}}, [\text{IDc} || \text{ADc} || \text{TS}_3])$$



# Kerberos Version 4 – Step-5, 6

- The client now decrypts the **TGS** response using the TGS session key.
- The client now has a session key for use with the new server, and a ticket to use with that server.
- The client can contact the new server using the same format used to access the **TGS**.

$$\text{Ticket}_v = E(K_v, [K_{c,v} || \text{ID}_c || \text{AD}_c || \text{ID}_v || \text{TS}_4 || \text{Lifetime}_4])$$

$$\text{Authenticator}_c = E(K_{c,v}, [\text{ID}_c || \text{AD}_c || \text{TS}_5])$$

# Ticket

---

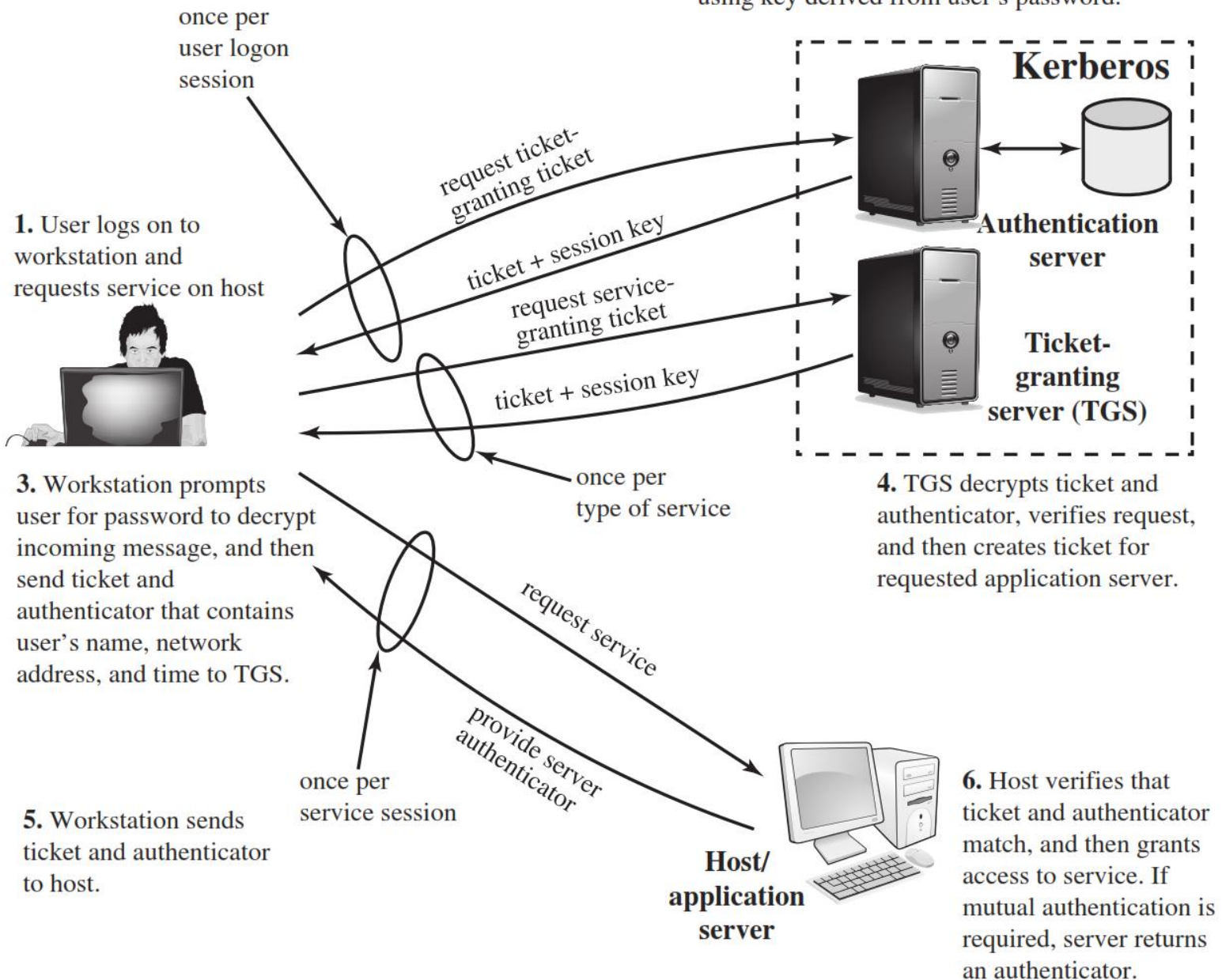
- Each request for a service requires a ticket.
- A ticket provides a single client with access to a single server.
- Tickets are dispensed by the “ticket granting server” (**TGS**), which has knowledge of all the encryption keys.
- Tickets are meaningless to clients, they simply use them to gain access to servers.
- The **TGS** seals (encrypts) each ticket with the secret encryption key of the server.
- Sealed tickets can be sent safely over a network - only the server can make sense out of it.
- Each ticket has a limited lifetime (a few hours).

# Ticket Contents

---

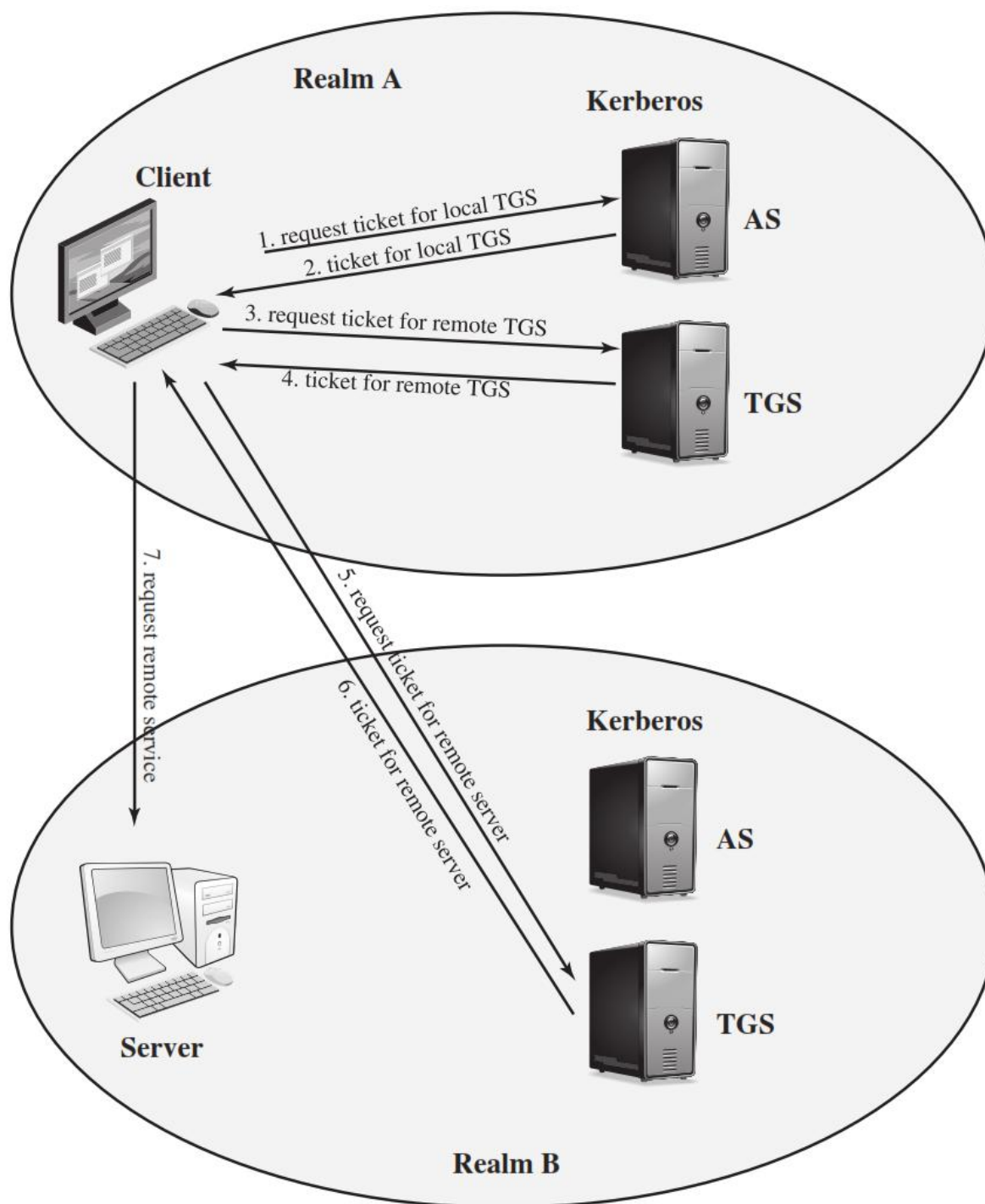
- Client name (user login name)
- Server name
- Client host network address
- Session key for client/server
- Ticket lifetime
- Creation timestamp







# Kerberos Realm



# Remote user authentication with Asymmetric encryption

---

## Mutual Authentication

1.  $A \rightarrow AS$ :  $ID_A \parallel ID_B$
2.  $AS \rightarrow A$ :  $E(PR_{as}, [ID_A \parallel PU_a \parallel T]) \parallel E(PR_{as}, [ID_B \parallel PU_b \parallel T])$
3.  $A \rightarrow B$ :  $E(PR_{as}, [ID_A \parallel PU_a \parallel T]) \parallel E(PR_{as}, [ID_B \parallel PU_b \parallel T]) \parallel E(PU_b, E(PR_a, [K_s \parallel T]))$

1.  $A \rightarrow KDC$ :  $ID_A \parallel ID_B$
2.  $KDC \rightarrow A$ :  $E(PR_{auth}, [ID_B \parallel PU_b])$
3.  $A \rightarrow B$ :  $E(PU_b, [N_a \parallel ID_A])$
4.  $B \rightarrow KDC$ :  $ID_A \parallel ID_B \parallel E(PU_{auth}, N_a)$
5.  $KDC \rightarrow B$ :  $E(PR_{auth}, [ID_A \parallel PU_a]) \parallel E(PU_b, E(PR_{auth}, [N_a \parallel K_s \parallel ID_B]))$
6.  $B \rightarrow A$ :  $E(PU_a, [E(PR_{auth}, [(N_a \parallel K_s \parallel ID_B))] \parallel N_b])$
7.  $A \rightarrow B$ :  $E(K_s, N_b)$



**THANK  
YOU FOR  
LISTENING  
ANY  
QUESTION ?**



**SUBSCRIBE**